HACKATHON BUILD SPECIFICATION / VERSION 1.0

VEILMINT

Evidence-Compiled Least-Data Firewall

FINAL IMPLEMENTATION PLAN

A 48-hour implementation blueprint for detecting unnecessary sensitive data in API flows, sanitising application logs before storage, and verifying the protected paths with synthetic canaries.

Build target
A demonstrable, deterministic prototype using a fictional NEXUS BANK scenario.

Development mode
Codex-assisted implementation from this specification. This document contains plans and acceptance criteria, not starter code.

4 NOVEMBER 2026 | 48-HOUR HACKATHON PLAN

01 / Product definition

What VEILMINT will actually do

Problem statement. Financial institutions handle large volumes of sensitive customer information across APIs, applications and system logs. The challenge is to identify sensitive customer information in API traffic and application logs and automatically protect or mask it before it is stored or shared.

VEILMINT discovers what a downstream consumer demonstrably needs, compiles that evidence into a destination-specific policy, enforces it at the API boundary and log pipeline, then checks the tested sinks for raw synthetic canary values.

The product loop

Step	What happens	Visible output
1. Discover	Load a producer schema and the Python consumer that receives its response.	Field inventory and source evidence.
2. Classify	Assign usage status and sensitivity as separate dimensions; preserve uncertainty.	USED / CANDIDATE_UNUSED / UNKNOWN plus sensitivity labels.
3. Compile	Build a versioned policy for the API destination and a separate policy for telemetry.	Reviewable policy bundle.
4. Review	A human approves only candidate-unused API fields for removal.	Approved removal list and policy history.
5. Enforce	Filter approved fields before the downstream consumer; sanitise log events before the demo sink.	Reduced API payload and protected log event.
6. Verify	Inject unique synthetic markers and inspect the final destination and log sink.	Pass/fail evidence for the tested paths.

The differentiator

The value is not simply “PII detection”. VEILMINT connects consumer-usage evidence to a least-data API contract, reuses destination-specific rules for telemetry, and then verifies the result. The defensible claim is workflow integration and evidence-backed enforcement in this prototype - not invention of DLP, redaction, static analysis or purpose-based access control.

What success looks like in the demo

- One click loads the deterministic NEXUS BANK example with 10 top-level fields.
- The analyser returns 5 USED, 4 CANDIDATE_UNUSED and 1 UNKNOWN, with evidence and scan limitations.
- A reviewer approves the four candidate fields; the API runtime sends a 6-field consumer payload and retains the UNKNOWN field.
- A structured application log is sanitised before it is written to the protected sink.
- A canary run proves that the tested raw markers are absent from the protected outputs while the expected pseudonymous correlation value remains.

02 / Scope and release boundary

The 48-hour MVP

Build one complete, honest golden path. The full product architecture can be represented, but only the components below need to be implemented to working quality during the hackathon.

P0 - required for a valid demo

Capability	MVP commitment	Acceptance evidence
Golden scenario	One fictional NEXUS BANK account-overview flow; fixed schema, fixture, consumer and reset.	Same clean state can be loaded repeatedly.
Schema inventory	JSON Schema parsing for top-level properties only.	All 10 expected fields and types are shown.
Consumer analysis	Python AST scan for literal subscript and .get access; conservative handling of dynamic access, reflection and forwarding.	Exact golden statuses; unsupported patterns become UNKNOWN or an incomplete-scan warning.
Sensitivity classification	Deterministic field-name/schema rules, with limited value-pattern checks where useful.	Each field has an explainable label and reason; no LLM dependency.
Policy review/compiler	Versioned policy; only CANDIDATE_UNUSED fields can be selected for API removal.	USED and UNKNOWN cannot be selected; empty approval is the default.
API protection	A controlled FastAPI route applies approved removals before returning/forwarding the consumer payload.	10-field original vs 6-field protected result; UNKNOWN stays.
Log protection	A structured logging filter/adapter applies DROP, MASK or HMAC pseudonymisation before writing to SQLite.	Stored event contains no configured raw synthetic values.
Canary verification	Unique synthetic markers are injected through the controlled API and log paths; the final outputs are inspected.	A repeatable pass/fail result identifies exactly which paths were checked.
Workbench and demo	A focused UI shows evidence, review, before/after and proof.	Complete demo from a clean reset.

P1 - only after the golden path passes

- Downloadable audit and policy report; simple policy history.
- Schema drift warning for an additional field not present in the compiled policy.
- Basic latency and transformation counters, reported as measured values only.
- An optional OpenTelemetry Collector demonstration if the API and log path are already stable.

Explicitly out of scope

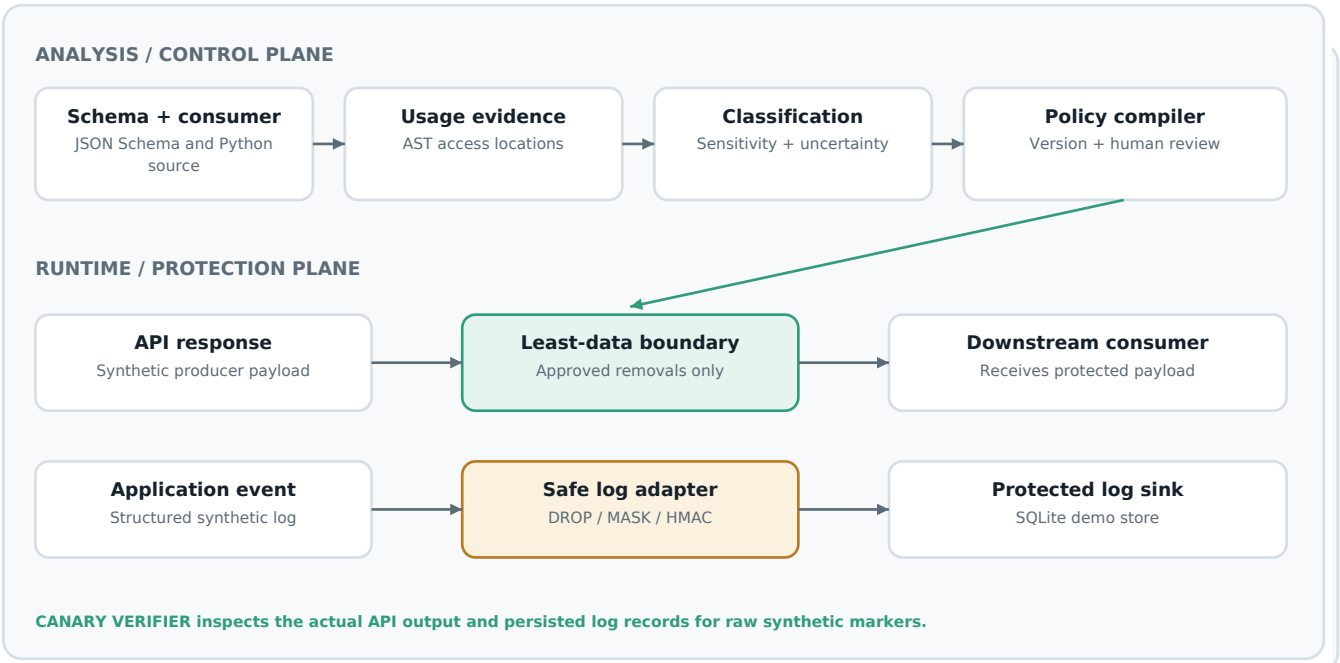
- Real bank integrations, real customer data, production credentials, login, account transfers or full banking workflows.
- Kubernetes, Istio, Envoy, OPA deployment, multi-tenant administration or a production policy service.
- LLM-based code reasoning, arbitrary source execution, nested JSON analysis, Java/Go/Node analysers or universal traffic interception.
- Historical log remediation, a production token vault, real Aadhaar/PAN/card data, or a claim of legal compliance certification.

Scope rule: do not start P1 work until the API protection path, pre-sink log sanitisation and canary proof all work together.

03 / Architecture

One analysis plane, two protection paths, one verification loop

Separate policy analysis from runtime enforcement. The analyser may be slow or unavailable without taking the controlled demo request path down. Runtime components consume a compiled policy and make deterministic



Analysis / control plane

Component	Responsibility	MVP implementation
Schema inventory	Read producer contract and enumerate fields/types.	Python + jsonschema.
Usage evidence engine	Find supported accesses in consumer code.	Python ast; never execute source.
Sensitivity classifier	Label fields using metadata, field names and limited detectors.	Deterministic rules; explainable reason.
Uncertainty engine	Prevent unsafe removal when scans are incomplete or ambiguous.	UNKNOWN status and warnings.
Contract compiler	Produce separate API and telemetry actions.	Versioned JSON policy document.
Review workbench	Show evidence and collect explicit approval.	React + TypeScript UI.

Runtime / protection plane

Path	Processing order	Boundary being protected
API traffic	Synthetic request -> FastAPI route/middleware -> approved field minimisation -> downstream consumer simulation.	The one controlled account-overview API-to-consumer path.
Application logs	Structured application event -> logging filter/adapter -> field transformation -> SQLite protected sink.	The one instrumented application logger and its configured sink.
Verification	Canary injection -> inspect API destination output and persisted log records -> record result.	Only the paths and fields exercised by the canary run.

Evidence ledger: policy decisions and test results only - never raw customer values or canary strings.

Freeze the data before building features

NEXUS BANK is a fictional environment used to make the problem visible. It is not a banking application. The same schema, fixture and consumer source must drive the analyser, runtime simulation, tests and judge demo.

Field	Expected usage	Sensitivity	Expected decision
account_id	USED	Internal identifier	Keep for API; HMAC pseudonym in telemetry.
account_type	USED	GENERAL	Keep for API; no log action unless required.
currency	USED	GENERAL	Keep for API; no log action unless required.
available_balance	USED	FINANCIAL	Keep for API; omit from logs unless needed.
recent_transactions	USED	FINANCIAL	Keep for API; do not log full values.
account_number	CANDIDATE_UNUSED	HIGH / BANK_ACCOUNT	Remove from consumer contract only after approval; DROP from telemetry.
customer_phone	CANDIDATE_UNUSED	MEDIUM / PHONE	Remove after approval; MASK or DROP in telemetry.
customer_address	CANDIDATE_UNUSED	MEDIUM / ADDRESS	Remove after approval; DROP in telemetry.
aadhaar_number	CANDIDATE_UNUSED	HIGH / GOV_ID	Remove after approval; DROP from telemetry.
internal_risk_flag	UNKNOWN	INTERNAL	Keep; removal disabled; surface uncertainty.

Analyzer behaviour

- Support top-level JSON Schema properties only in the MVP. Nested objects should be marked unsupported/incomplete, not silently treated as scanned.
- Recognise literal access patterns such as payload["account_id"] and payload.get("account_id"). Record filename, line and access kind.
- Dynamic key access, reflection, broad object forwarding, aliasing the analyser cannot resolve, and incomplete files must raise an uncertainty warning. Do not convert ambiguity into CANDIDATE_UNUSED.
- For the golden fixture, implement the smallest field-sensitive dynamic-access/provenance rule necessary to preserve the intended single UNKNOWN field. Do not use a hardcoded result table as a substitute for scanning the consumer.
- No import, exec, eval, subprocess, filesystem access or execution of uploaded source. Parsing limits and file-size limits apply.

Golden acceptance result

5 USED | 4 CANDIDATE_UNUSED | 1 UNKNOWN

A change to the consumer, schema or expected statuses must update the golden test deliberately. The same input and rule version should produce the same findings.

Sensitivity and usage remain separate

A field can be USED and HIGH sensitivity at the same time. “Used” does not mean safe for logs, and “sensitive” does not by itself mean removable from the API. Each destination gets its own policy action.

The safety invariants

The policy bundle is the contract between analysis and runtime. Keep it versioned, reviewable and deterministic. The same field may be allowed for business processing but prohibited or transformed in telemetry.

Finding	API action	Telemetry action
USED	KEEP / ALLOW. Never auto-remove based on usage scan.	Apply telemetry rule independently; raw value is not automatically allowed.
CANDIDATE_UNUSED	KEEP by default. REMOVE only after explicit reviewer approval and passing contract validation.	DROP or MASK according to sensitivity policy.
UNKNOWN	KEEP; removal disabled; raise warning.	Apply sensitivity-based DROP / MASK / pseudonymisation.
New/unmapped field	In MVP, retain and alert in observe/warn behaviour; do not silently drop.	Do not emit raw if covered by a known rule; otherwise flag as unclassified and record the limitation.

Telemetry transformation actions

Action	Meaning	Example use
DROP	Remove the attribute from the event entirely.	Aadhaar marker, account number, address.
MASK	Replace the value with a fixed redaction marker.	Phone value where presence is useful but the original is not.
HMAC_PSEUDONYM	Replace value with a keyed, stable pseudonym for correlation.	account_id in approved operational telemetry.
ALLOW	Retain only when a policy explicitly permits the attribute for that sink.	Non-sensitive operational event metadata.

Required compiler invariants

- USED => never automatically remove.
- UNKNOWN => never automatically remove.
- CANDIDATE_UNUSED => removal requires explicit approval.
- The default approval selection is empty.
- Every removal must be linked to the audit ID, policy version and reviewer action.
- The API allow/remove policy and telemetry transform policy are distinct objects/sections.
- A missing rule must not cause silent field deletion. During the hackathon, retain and flag unrecognised API fields.

Policy bundle fields

```
policy_id, version, created_at, producer, consumer, operation, mode, schema_hash, source_hash, compiler_version,
reviewer, fields{usage, sensitivity, api_action, evidence}, telemetry{field: action}
```

Runtime modes

Mode	Behaviour	Hackathon use
OBSERVE	Measure/report proposed changes; do not apply API removals.	Show baseline exposure.
WARN	Pass the request but surface policy drift or a violation.	Fallback for unknown/new API fields.
ENFORCE	Apply only the approved, compiled API removals and telemetry transformations.	Controlled golden demo after review.

06 / Technical implementation

Stack and service contracts

Layer	Hackathon choice	Boundary / reason
Backend	Python 3.12+; FastAPI; Pydantic	One language for AST analysis and API runtime; typed contracts.
Schema validation	jsonschema	Validate source schema and reduced contract against synthetic fixtures.
Static analysis	Python ast	Deterministic parsing; analysed code is never executed.
Frontend	React + TypeScript + Vite	Fast iteration and typed API client.
UI system	Tailwind CSS + shadcn/ui; Motion for React	Premium, restrained workbench; reduced-motion support.
Persistence	SQLite	Local demo policies, audit records and protected log sink.
Telemetry	Python structured logging; optional OTel Collector later	Use a single known pre-sink adapter for the MVP.
Packaging / CI	Docker Compose; GitHub Actions	One-command local run; backend tests and frontend build.

Minimum backend routes

Route	Purpose
GET /api/scenarios	List available demo scenarios.
POST /api/scenarios/nexus-bank/load	Reset/load schema, fixture, consumer and expected demo metadata.
POST /api/audits	Run schema + consumer analysis.
GET /api/audits/{id}/fields	Return per-field usage, sensitivity, evidence and warnings.
POST /api/audits/{id}/proposal	Create a proposed minimum contract from explicit review selections.
POST /api/audits/{id}/validate	Validate proposed contract against the synthetic fixture.
POST /api/policies/compile	Compile immutable API and telemetry policy sections.
POST /api/runtime/demo/request	Run the controlled API-to-consumer simulation using the active policy.
POST /api/runtime/demo/log-event	Send a structured synthetic application event through the safe logger.
POST /api/verification/canary	Run the canary experiment across controlled destinations.
GET /api/verification/{run_id}	Return checks, scope, results and evidence.
GET /healthz	Health check for local/deployed environment.

Repository structure

```
veilmint/
backend/app/{api,analyzer,compiler,runtime,verification,models}/
frontend/src/{app,components,features,lib}/
demo/nexus-bank/{schema,response,consumer,scenario}/
policies/
tests/{unit,integration,security,golden}/
scripts/reset_demo.py
docker-compose.yml
.github/workflows/ci.yml
README.md .env.example
```

Source of truth

Freeze the Pydantic request/response models, policy model and TypeScript types before parallel implementation. Generate or maintain one shared API contract. Do not let the frontend invent field statuses or policy results.

07 / Workbench implementation

Show the evidence, not a generic security dashboard

Visual direction: premium dark API-forensics workbench. Keep the NEXUS BANK context small and the security workflow central. No fake SOC widgets, invented traffic volume, decorative risk gauges or simulated scan progress.

Screen / state	What it contains	Primary visual moment
Scenario overview	NEXUS BANK account overview; producer, consumer, operation; original 10-field response.	Expose the over-broad payload and destinations.
Field findings	One row per field; usage status, sensitivity label, evidence location, warning and allowed action.	Field Evidence Map: field-to-source relationship.
Policy review	Candidate-unused fields selectable; USED/UNKNOWN disabled; separate API and telemetry actions.	Reviewer can see exactly what is being approved.
Contract result	Original vs consumer payload; reduced schema and policy version.	Schema Diff / 10 fields to 6 fields.
Runtime trace	Request ID, policy version, original field count, transformed result, downstream consumer receipt.	Show enforcement happened before handoff.
Log protection	Structured event before adapter (synthetic only) and persisted safe event.	Show DROP / MASK / HMAC action per field.
Verification	Canary run ID, paths checked, raw-match result, expected pseudonym result and limitations.	Validation Reveal: evidence of non-leakage for tested paths.

Design system

Token	Value / use
Canvas	#0C1017
Panel / raised	#141B25 / #1C2733
Border	#303C49
USED	#55C7A5
CANDIDATE_UNUSED	#E9AE58
UNKNOWN	#9AA8B8
Typography	Inter or Geist for UI; JetBrains Mono for code and identifiers.
Components	shadcn/ui + Tailwind; Motion for React for restrained transitions.

Interaction rules

- Usage status and sensitivity are independent columns/tags; never merge them into one risk badge.
- Every field decision opens evidence or an explicit reason; no unexplained score.
- The review action is impossible for USED and UNKNOWN fields.
- Show loading, empty, error and reset states. Avoid fake progress bars.
- Use a custom evidence relationship view and schema diff; do not add React Flow for this one small scenario.
- Respect reduced motion. Use code blocks as read-only evidence, not as an editor.

08 / 48-hour execution schedule

Build in vertical slices and integrate every 2-3 hours

The timeline is a sequencing target, not a promise that every optional item will be completed. The lead should cut polish and secondary features before compromising the safety invariants or the integrated demo.

Time	Workstream	Required output / exit gate
0-2 h	Repo freeze	Monorepo, owners, branch strategy, CI skeleton, shared API/policy contract.
2-5 h	Golden scenario	NEXUS schema, fixture, Python consumer, scenario manifest, reset script, fixture tests.
5-9 h	Schema + AST	Top-level schema parser; literal access detection; evidence with filename/line; uncertainty warnings.
9-14 h	Sensitivity + uncertainty	Rule-based labels; targeted dynamic-access handling; exact golden statuses.
14-17 h	Golden test gate	Tests freeze the 5/4/1 result and the safety invariants before policy work expands.
17-22 h	FastAPI contracts	Scenario, audit, fields, proposal and validation endpoints; frontend can consume real results.
22-26 h	Workbench vertical slice	Scenario, findings, evidence interaction and policy review screen.
26-29 h	Policy compiler	Separate API and telemetry rules; immutable version; reviewer approval recorded.
29-32 h	API runtime	Before/after request path; remove only approved candidates; retain UNKNOWN and unmapped fields with warning.
32-35 h	Safe logging	Structured log adapter; DROP/MASK/HMAC; persist only transformed event to SQLite sink.
35-37 h	Canary verification	Inject markers, inspect actual API output and persisted log sink, store run result.
37-40 h	Evidence UI	Policy trace, log transform view and canary reveal wired to actual backend output.
40-42 h	Packaging	Docker Compose clean start; README; environment example; health check.
42-44 h	Integration gate	Full golden-path test from reset through API and log verification.
44-46 h	Polish / fix	Error states, reset, response clarity, no new features unless critical.
46-48 h	Rehearsal	2-3 minute demo, backup recording/screenshots, clean-room run and judge Q&A.;

Hard checkpoint rules

- At hour 5, the scenario must load from a clean reset.
- At hour 17, golden analysis and critical safety tests must pass.
- At hour 22, the backend contract must be callable by the frontend.
- At hour 35, API minimisation and log sanitisation must both work without the UI.
- At hour 37, the canary verifier must inspect real output records, not a hardcoded PASS response.
- At hour 44, stop feature work. Only repair, polish and rehearse.

Parallel workstreams with one integration owner

Use four ownership lanes even if individual people cover more than one lane. Each lane should have a narrow contract and a testable deliverable. Do not let separate Codex sessions invent incompatible request or policy models.

Owner lane	Owns	Must hand over
A. Analysis engine	Schema parser, Python AST scan, uncertainty model, sensitivity rules.	Stable finding model; golden analyser tests; evidence and warnings.
B. Backend + runtime	FastAPI routes, policy compilation, API minimiser, safe logging, SQLite sink.	OpenAPI contract; runtime trace; transformed event records.
C. Frontend workbench	Scenario UI, field table/evidence map, approval UI, diff, log trace, verification reveal.	Typed API client; responsive demo path; no fabricated state.
D. Scenario + QA + delivery	Golden fixture, synthetic data, test design, canary test, Docker Compose, CI, README and demo rehearsal.	Deterministic reset; integration tests; local-run fallback; backup media.

Codex operating protocol

1. Freeze contracts first. Agree field enums, sensitivity labels, policy schema, endpoint payloads and folder ownership before parallel coding.
2. Give Codex bounded tasks. Each task names exact files, inputs, outputs, invariants, tests and forbidden side effects. Ask it to inspect existing files before changing them.
3. One issue per task. Avoid asking a coding agent to redesign the architecture while implementing a feature.
4. Require tests with every change. Analyzer changes update golden tests; policy changes test USED/UNKNOWN protection; runtime changes test actual stored output.
5. Integrate frequently. Merge or reconcile every 2-3 hours. Keep main runnable; avoid long-lived isolated branches.
6. Review generated changes. Inspect diffs, run tests, verify imports/dependencies and check for unsafe execution, logging leaks or fabricated UI values.
7. Keep a visible decision log. Record any deviation, why it was needed, what acceptance criterion changed and who approved it.

Git and release practice

- Small commits with one purpose; protect the main branch from untested integration.
- Pin dependencies and keep secrets out of Git. Commit only synthetic fixtures and safe configuration examples.
- CI gates: backend tests, frontend typecheck/build, golden results, policy invariants and a golden demo smoke test.
- Keep one tagged demo candidate and a known-good local environment before the final rehearsal.

A pass is based on observed outputs

Test layer	Required test	Release gate
Unit	AST literal subscript and .get recognition; line evidence; field rule classification; HMAC and redaction helpers.	Critical rule branches covered.
Golden	NEXUS BANK schema + consumer produces 5 USED, 4 CANDIDATE_UNUSED, 1 UNKNOWN.	Exact deterministic match.
Negative / uncertainty	Dynamic access, reflection, broad forwarding, unsupported nested fields and truncated/incomplete scan.	Never silently become CANDIDATE_UNUSED.
Policy safety	Attempt to select USED or UNKNOWN for removal; compile with no approvals; submit unknown field.	Selection rejected; defaults safe; unknown field retained and warned.
Contract	Reduced schema validates the expected downstream fixture and preserves required properties.	Validation passes before ENFORCE mode.
API runtime	Actual controlled request is processed under policy; only approved candidates removed.	Five USED plus one UNKNOWN retained; four approved candidates removed.
Logging runtime	Sensitive structured event passes through filter before SQLite insert.	Persisted record has no raw values covered by the test policy.
Canary	Unique canary values injected; verifier inspects final API output and stored log rows.	No raw match in protected outputs; expected pseudonym check succeeds where configured.
Integration	Reset, analyse, review, compile, API request, log event and verification run in one workflow.	Complete clean-start demo passes.

Demo acceptance numbers

Measure	How to report it
Field reduction	Show 10 original fields -> 6 consumer fields for the fixed golden scenario.
Serialized size	Measure UTF-8 JSON bytes before and after using the actual fixture; display the measured values only.
Sanitisation	Count actual DROP, MASK and HMAC transformations in the specific log event.
Verification coverage	List paths/fields that were injected and checked; do not describe it as universal coverage.
Repeatability	Run from clean reset more than once and compare deterministic findings/policy decisions.

Release gate: every P0 acceptance criterion must pass, the full demo must work from a clean start, and the team must be able to state exactly what the canary did and did not test.

Make the prototype safe and make its claims precise

VEILMINT processes source code, field metadata and synthetic payloads. The prototype must not become another place where raw customer information is copied or retained.

Mandatory safety controls

- Synthetic data only. Use obvious test identifiers and non-operational canary markers; never generate or import real customer identifiers.
- Never execute analysed source. Python AST parsing only; apply file-count, aggregate-size, UTF-8, AST-node and timeout limits.
- Do not log raw request/response bodies in application logs. The demonstration may show a synthetic preview, but the protected sink only receives transformed events.
- Keep the HMAC secret in environment configuration, never in source control. Do not claim production key management.
- The evidence ledger stores metadata and outcomes, not raw PII or raw canary values.
- Protect policy changes with reviewer identity in the demo and immutable policy versioning. Digital signing/RBAC are production extensions, not hackathon claims.

Risk register

Risk	Why it matters	MVP mitigation
False unused finding	A required property could be removed and break a consumer.	UNKNOWN on ambiguity; human approval; contract fixture test.
Dynamic or hidden access	AST scan may not see reflection, aliases or runtime-generated keys.	Explicit unsupported-pattern warnings; conservative status; disclose coverage.
Sensitive detector miss	A field may not match a known rule.	Schema/name rules for the frozen demo; canary checks only known test markers.
Logging bypass	Direct print/file writes or uninstrumented handlers may bypass the adapter.	Limit claim to the configured application logger and sink; test that path.
Policy drift	Schema or consumer changes can stale the bundle.	Store schema/source hashes; show mismatch warning.
Sink verifier overclaim	A negative scan cannot prove no leak anywhere else.	Show exact sinks, fields and run ID checked; no universal guarantee.
Scope explosion	Gateway, OTel, multi-language and production features consume the build window.	P0-only gate; cut P1 polish before core proof.

Claims we may make

“In our NEXUS BANK prototype, VEILMINT derives a conservative consumer-specific field proposal, enforces reviewer-approved removals on the controlled API path, sanitises the instrumented structured log path before SQLite storage, and verifies those test paths using synthetic canaries.”

Claims we must not make

- “VEILMINT prevents all sensitive data leakage” or “guarantees no data can leak”.
- “Works on every API, microservice, log platform or programming language.”
- “Automatically knows a field is unnecessary” without evidence, uncertainty handling and approval.

A 2-3 minute end-to-end story

Time	Action	What the judge sees / hears
0:00-0:20	Open NEXUS BANK account overview.	“This service sends ten fields to a consumer that needs only a subset.” Show the synthetic response.
0:20-0:45	Run analysis.	5 USED, 4 CANDIDATE_UNUSED, 1 UNKNOWN; open one source evidence item and one uncertainty warning.
0:45-1:05	Review and approve the four candidates.	Show that USED and UNKNOWN are locked. Compile the destination-specific policy.
1:05-1:30	Run the API request under enforcement.	Side-by-side 10-field source vs 6-field consumer payload. UNKNOWN remains.
1:30-1:55	Emit a structured application event.	Show field-specific DROP / MASK / HMAC actions and the sanitised persisted record.
1:55-2:20	Run canary verification.	Show raw marker not found in the tested API output and log sink; expected pseudonym found if configured.
2:20-2:40	Close with the differentiator.	“We do not only label sensitive data. We derive what a consumer needs, enforce that boundary, and verify the instrumented sinks.”

Demo preparation checklist

- The first screen opens preloaded; no empty upload form or setup delay.
- One reset action restores schema, consumer, fixture, policy and sink state.
- Use a clean policy version and a fresh canary run for the live demo.
- Keep the raw synthetic fixture, expected findings and expected protected output available for recovery.
- Prepare a local run and backup screen recording/screenshots. Rehearse the exact click sequence and speaker transitions.
- If deployment fails, run the same complete golden path locally; do not substitute a mock PASS screen.

Judge questions to prepare for

Question	Answer direction
Why not just use a DLP scanner?	DLP labels values; VEILMINT also derives consumer-specific minimum fields and links those findings to an enforceable contract.
How do you avoid breaking consumers?	Conservative UNKNOWN state, approval gate, contract validation and gradual observe/warn/enforce progression.
What if Python uses dynamic keys?	Unsupported/dynamic patterns are marked uncertain; this MVP does not treat an incomplete scan as proof a field is unused.
Does canary PASS prove no leak anywhere?	No. It proves absence only in the instrumented API output and log sink inspected in that run.
Is this production-ready/compliant?	No. It is a prototype of a control workflow; production needs broader coverage, security review, policy governance and deployment integrations.

Production evolution - not part of the 48-hour build

Stage	What to add	Why it comes later
1. Coverage and evidence	More precise object provenance, nested field support, multiple consumer repos, runtime usage evidence and schema drift reconciliation.	Reduces false-unused risk before widening enforcement.
2. Policy lifecycle	RBAC, multi-owner approvals, signed policy bundles, version rollback, staged rollout and audit retention.	Makes policy changes governable.
3. Runtime integrations	API gateway/Envoy integration, OpenTelemetry Collector processor, language-specific safe logging adapters and SIEM connectors.	Extends beyond the controlled FastAPI route and local sink.
4. Enterprise detection	Configurable detectors, structured/unstructured log support, encoding/normalisation, secrets and custom identifiers.	Broadens detection coverage with measured precision/recall.
5. Resilience and operations	Last-known-good policy cache, metrics, alerting, performance testing, bypass inventory and incident runbooks.	Addresses production availability and operational risk.
6. Governance	Purpose metadata, retention/consent controls, legal/security review and jurisdiction-specific mapping.	Supports organisational policy; does not replace legal compliance review.

Final decisions to keep locked

- Product: VEILMINT - Evidence-Compiled Least-Data Firewall.
- Demo context: synthetic NEXUS BANK account-overview API; no live bank integration.
- Core architecture: evidence analysis -> destination-specific policy -> API enforcement + pre-sink log sanitisation -> independent canary verification.
- MVP stack: Python/FastAPI, Python AST, JSON Schema, React/TypeScript/Vite, Tailwind/shadcn/ui, SQLite and Docker Compose.
- Safety: no source execution; no automatic removal of USED or UNKNOWN; explicit review for candidate-unused API fields; safe default for unmapped fields.
- Proof: test actual output and sink records; report the exact scope tested.

Basis of this plan

This plan consolidates the uploaded Engineering Research Report on Veilgate, API Minimizer and VEILMINT; the Prevent Sensitive Data Leakage problem analysis; the existing Stage 6-7 product/build specifications; the NEXUS BANK Stage 8.1 golden-demo bundle; the Stage 8 execution playbook; and the visual presentation strategy. Where the older API Minimizer documents conflict with VEILMINT's broader problem fit, this plan retains their useful analyser, review and golden-scenario mechanics but adds the minimum runtime API and pre-sink log-protection loop.

Implementation status

Planning specification only. No code, runtime integration, test result, deployment or security guarantee is represented as already completed.